

TOPA 3급 온라인

04 EDA



목차

—

EDA의 개념과 함께
다양한 데이터 시각화
방법과 데이터 전처리
방법을 알아봅시다.

01 데이터 시각화

02 **Matplotlib**

03 **Seaborn**

04 데이터 비교

05 상관관계 분석



목차

—

EDA의 개념과 함께
다양한 데이터 시각화
방법과 데이터 전처리
방법을 알아봅시다.

06 상관관계 시각화

07 데이터 스케일링

08 데이터 전처리

09 특성 엔지니어링

01

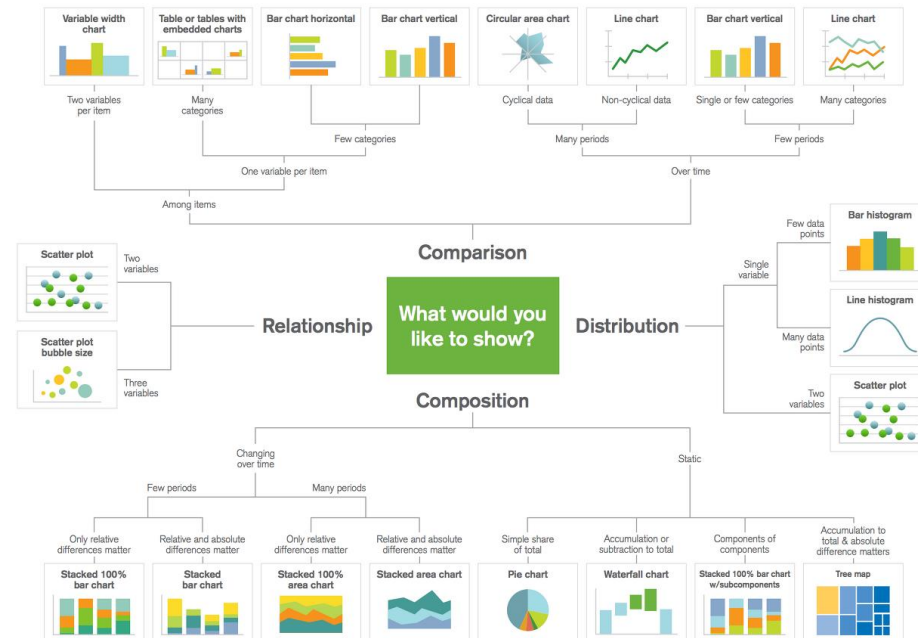
데이터 시각화

④ 탐색적 데이터 분석 (EDA: Exploratory Data Analysis)

- 수학자 존 튜키가 제안한 데이터 분석 방법으로, 데이터를 다양한 방법으로 확인하고 이해하는 과정
- 시각화 등의 방법을 통해 자료를 직관적으로 바라볼 수 있게 함
- 이 과정으로 도출된 데이터의 지식이 이후 통계 분석이나 모델링에 사용됨

☑ 데이터 시각화

- 그래프, 차트 또는 맵과 같은 시각적 요소를 사용해 데이터를 나타내는 프로세스
- 시각적인 포맷을 통해 수치 데이터로는 발견하지 못한 패턴 등의 인사이트를 발견하도록 도와줌



Source: GA, Abela, 2010. www.ExtremePresentation.com

☑ 데이터 시각화 방법

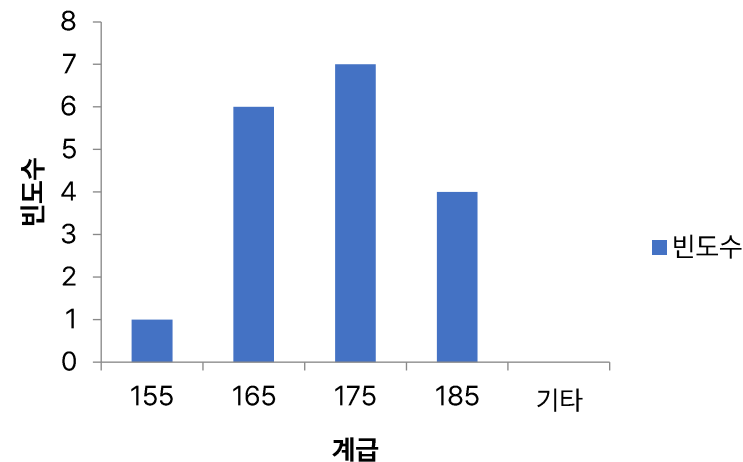
- 히스토그램(histogram): 단일 수치형 변수의 분포를 나타내는데 사용
- 박스 플롯(box plot): 이상치(outlier)를 식별하고 데이터의 분산을 파악하는데 사용
- 산점도(scatter plot): 두 수치형 변수 사이의 관계를 시각화 하는데 사용
- 막대 차트(bar plot): 범주형 변수의 카테고리 빈도나 비율을 표현하는데 사용
- 히트맵(heat map): 변수 간의 상관관계를 시각화 하는데 사용

④ 히스토그램(histogram)

- 표 형태의 도수 분포를 그래프로 표현한 것
- 일반적으로 가로축이 계급을 나타내며 세로축이 도수(개수)를 나타냄

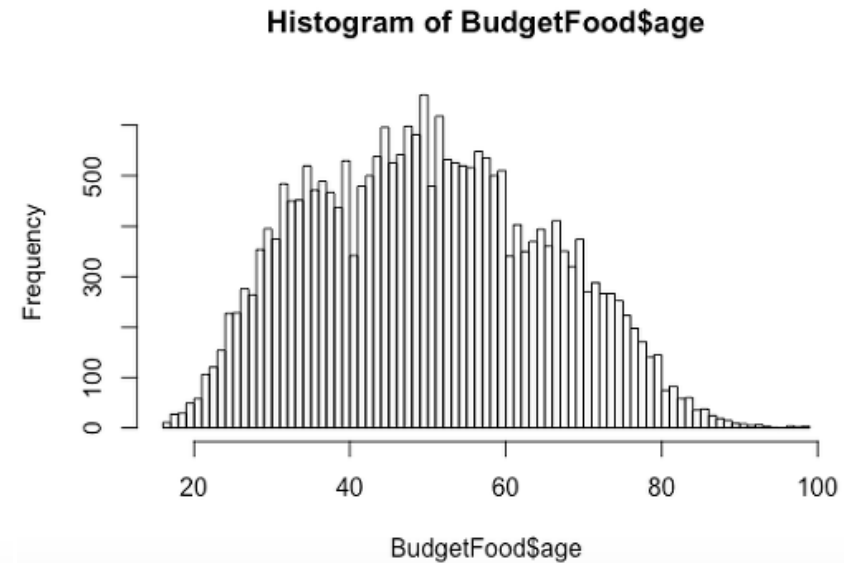
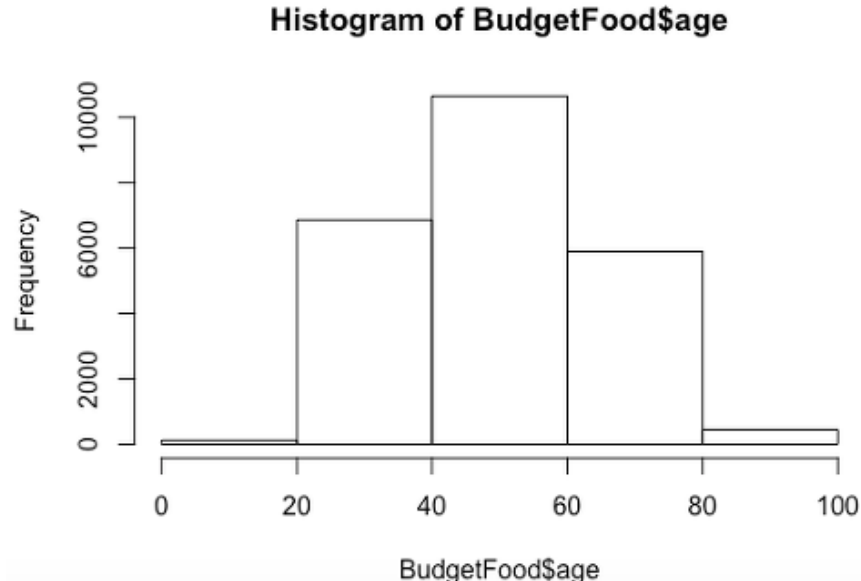
158	183	172	163	165	170
172	170	180	158	179	169
164	175	188	147	173	156

계급 구간	계급 구간 의미	도수
155	155이하	1
165	155초과 165이하	6
175	165초과 175이하	7
185	175초과 185이하	4



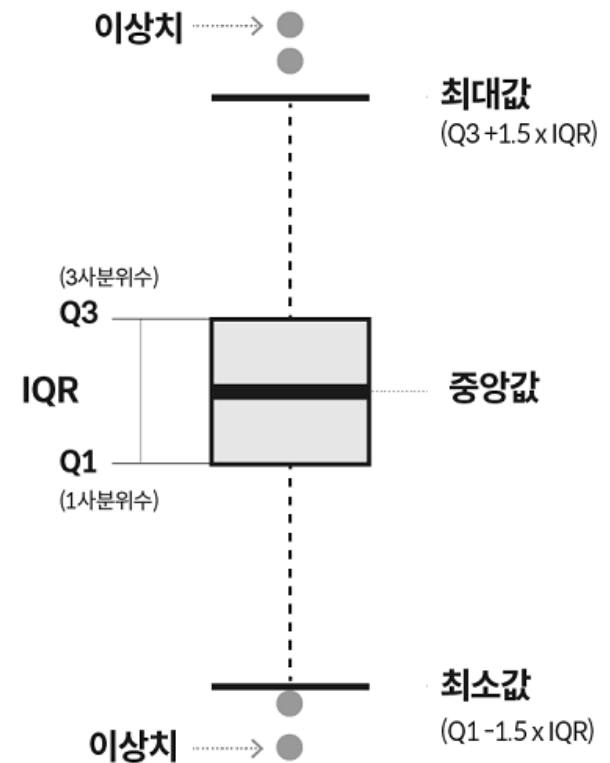
☑ 히스토그램(histogram)의 장점과 한계

- 장점: 분포를 시각화하기 때문에 각 계급에 속하는 자료의 수가 많고 적음을 쉽게 알 수 있음
- 한계: 원래의 값을 상실할 수 있으며, 연속적인 데이터에서만 사용이 가능함



☑ 박스 플롯(box plot)

- 데이터 분포와 이상치를 동시에 보여주는 시각화 방법
- 제1사분위수 (Q1): 데이터의 25% 지점의 값
- 중앙값: 데이터 세트의 절반을 분리하는 값
- 제3사분위수 (Q3): 데이터의 75% 지점의 값
- 최대값: $Q3 + 1.5 \times IQR$
- 최소값: $Q1 - 1.5 \times IQR$

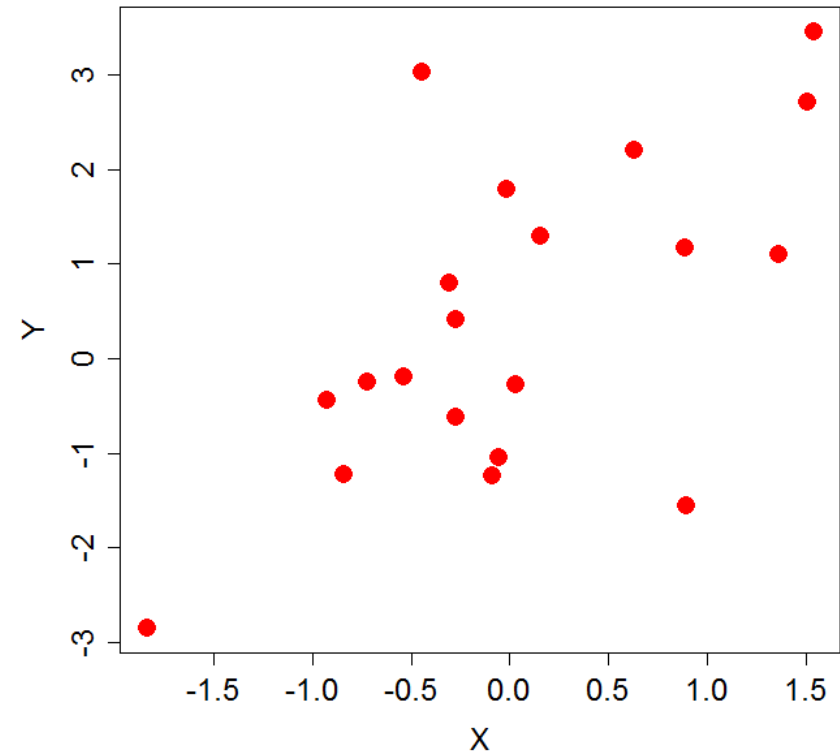


☑ 박스 플롯(box plot)의 장점과 한계

- 장점: 1. 분포의 간결한 시각적 요약
 - 2. 그룹 또는 카테고리 간 비교 가능
 - 3. 이상치와 잠재적 데이터 오류 강조
- 한계: 1. 기본 분포를 자세히 보여주지 못함

✓ 산점도(scatter plot)

- 두 변수사이의 관계, 추세, 상관 관계 등을 시각화 하는데 주로 사용

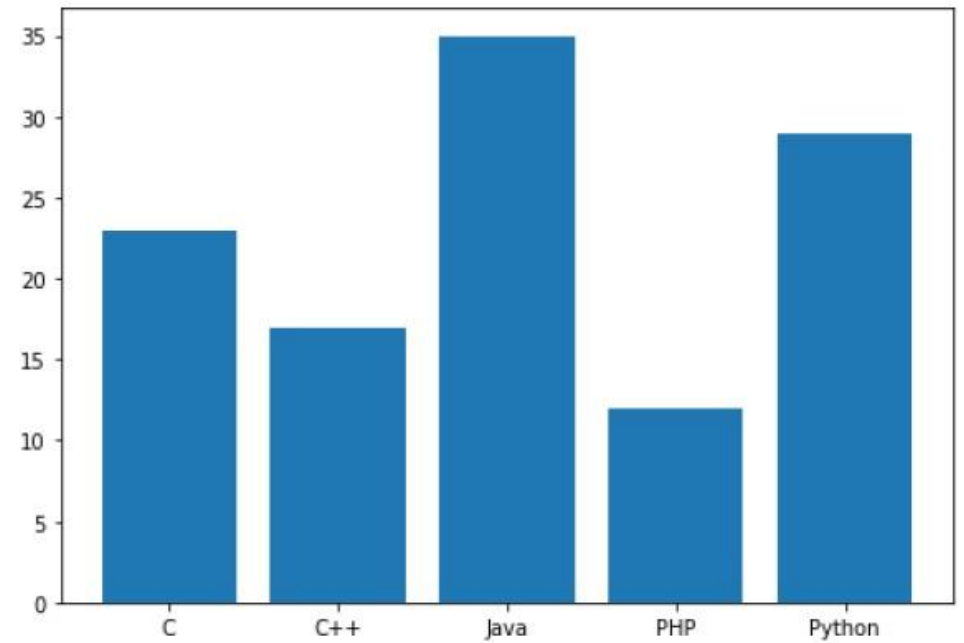


☑ 산점도(scatter plot)의 장점과 한계

- 장점: 1. 생성과 해석이 쉬움
 - 2. 두 변수 사이의 관계를 시각화 함
 - 3. 데이터의 패턴, 추세 및 이상치를 보여줌
- 한계: 1. 수치 데이터에 주로 적용 가능하며 범주형 데이터에는 적합하지 않음
 - 2. 매우 많은 수의 포인트를 가진 데이터 세트에서는 관계를 시각화하기 어려울 수 있음

☑ 막대 차트(bar plot)

- 그룹 또는 카테고리 간의 비교를 하는데 용이하게 사용

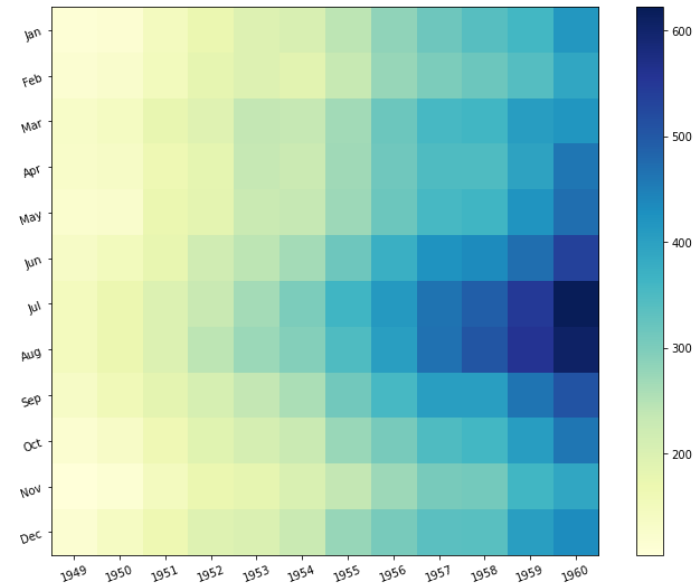
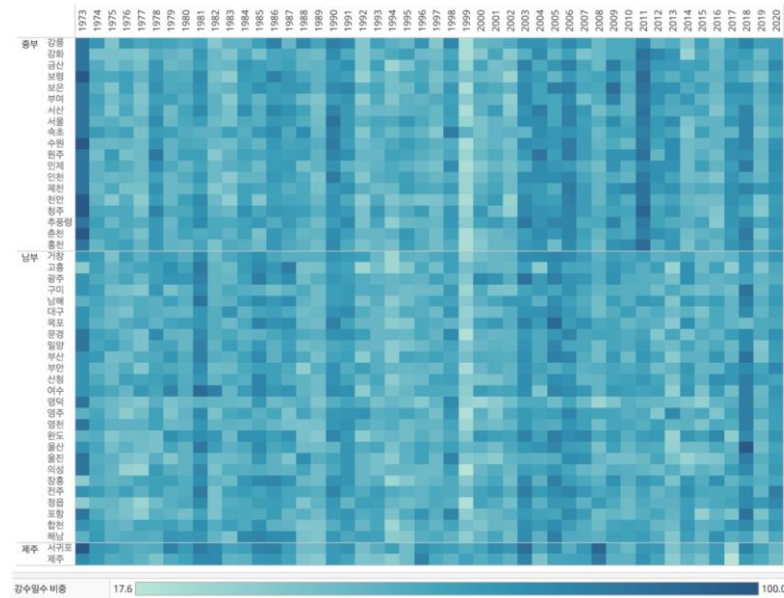


✓ 막대 차트(bar plot)의 장점과 한계

- 장점: 1. 해석이 쉬움
2. 이산 카테고리 간의 비교를 효과적으로 보여줌
- 한계: 1. 연속 데이터에는 적합하지 않음
2. 많은 양의 카테고리를 다룰 때 혼잡해질 수 있음

✓ 히트맵(heat map)

- 열을 뜻하는 히트(heat)와 지도를 뜻하는 맵(map)을 결합 시킨 단어
- 데이터의 값을 열 분포 형태로 표현



☑ 히트맵(heat map)의 장점과 한계

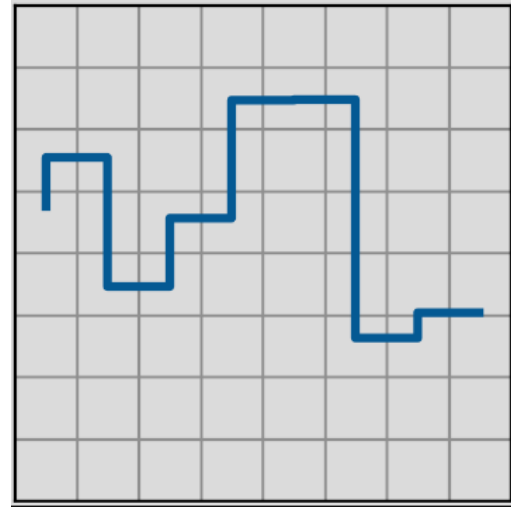
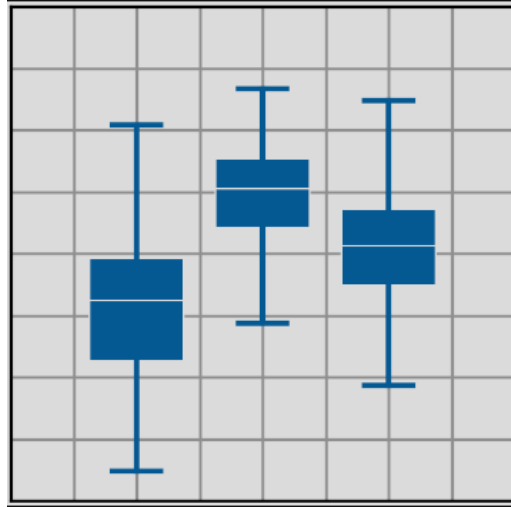
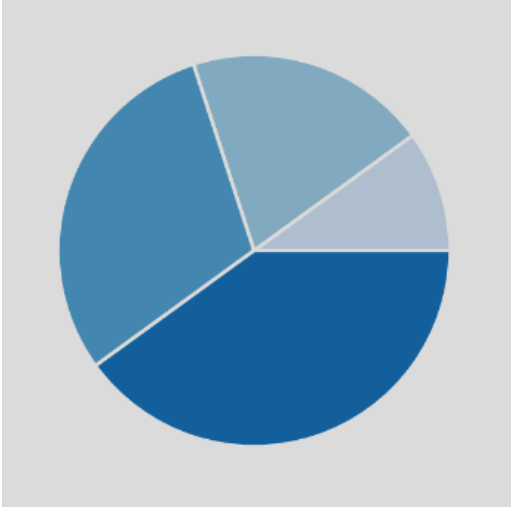
- 장점: 1. 시각적으로 직관적
 - 2. 대규모 및 복잡한 데이터셋을 다루는 데 효과적
 - 3. 빠르게 패턴, 군집, 상관 관계를 식별하는 데 유용함
- 한계: 1. 색상 척도의 선택이 신중하지 않으면 해석이 잘못될 가능성이 있음
 - 2. 색상 표현이 때로는 모호할 수 있어 자세한 수치 분석에는 적합하지 않을 수 있음

02

Matplotlib

✓ Matplotlib

- 파이썬 프로그래밍에서 널리 사용되는 데이터 시각화 라이브러리
- 다양한 데이터를 시각화 하는데 유용함



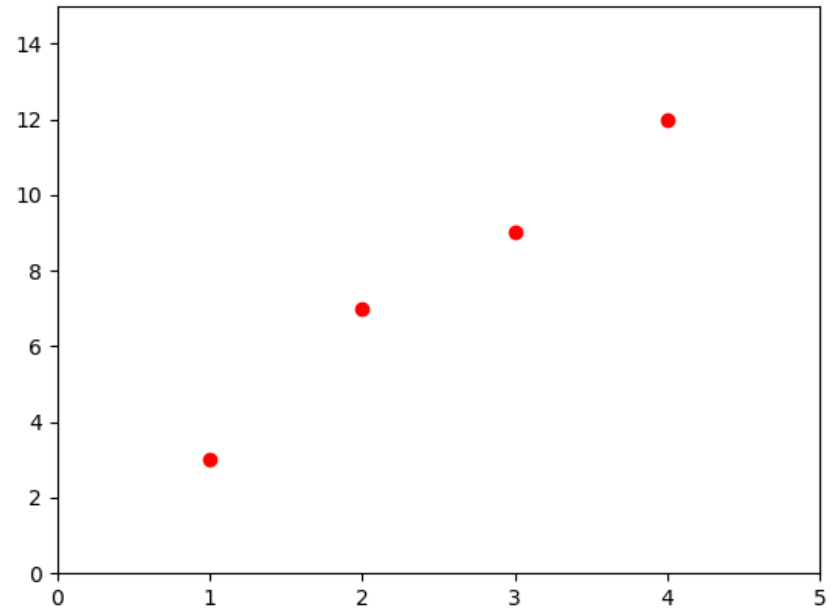
✓ Matplotlib

- (1, 3), (2, 7), (3, 9), (4, 12)인 점을 그래프에 나타내기

```
import matplotlib.pyplot as plt

# 플롯 그리기
# 'ro'는 빨간색('red') 원형 마커('o')를 의미함
plt.plot([1, 2, 3, 4], [3, 7, 9, 12], 'ro')
plt.axis([0, 5, 0, 15])
plt.show()
```

Result



☑ Matplotlib

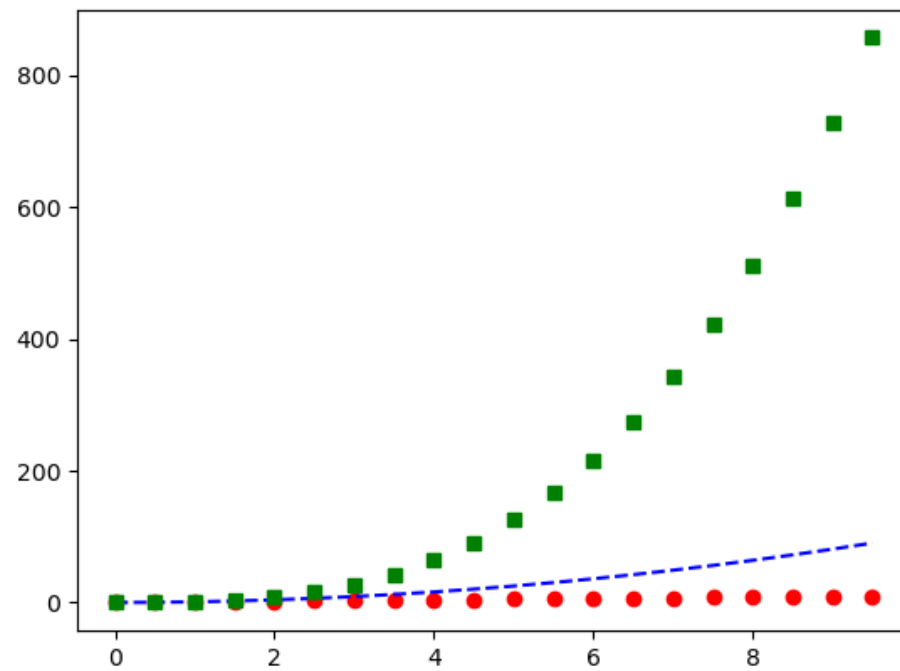
- 여러개의 그래프를 한번에 나타낼 수 있습니다.
- Time = [0, 0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5, 5.5, 6, 6.5, 7, 7.5, 8, 8.5, 9, 9.5]

```
import numpy as np
import matplotlib.pyplot as plt

time = np.arange(0, 10, 0.5)

plt.plot(time, time, 'ro', time, time**2, 'b--', time, time**3, 'gs')
plt.show()
```

Result



✓ Matplotlib의 한계

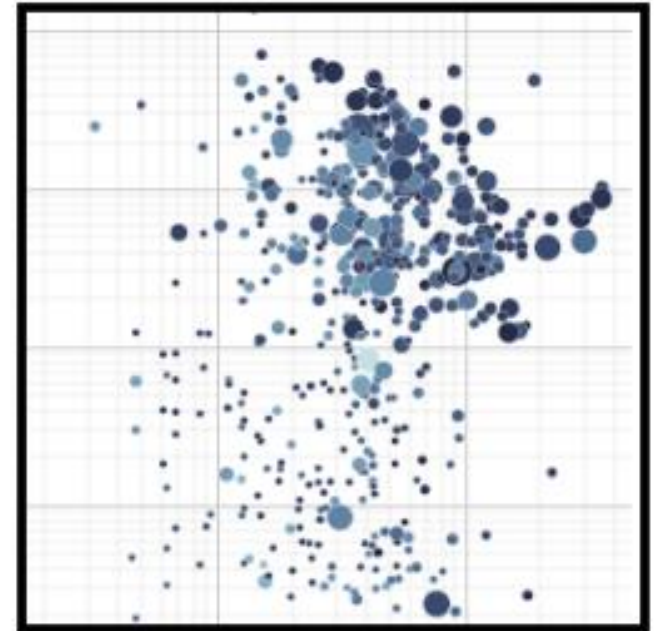
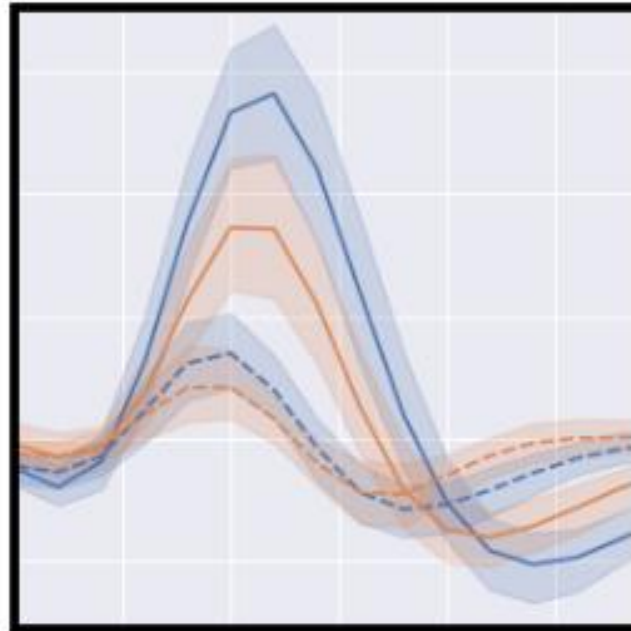
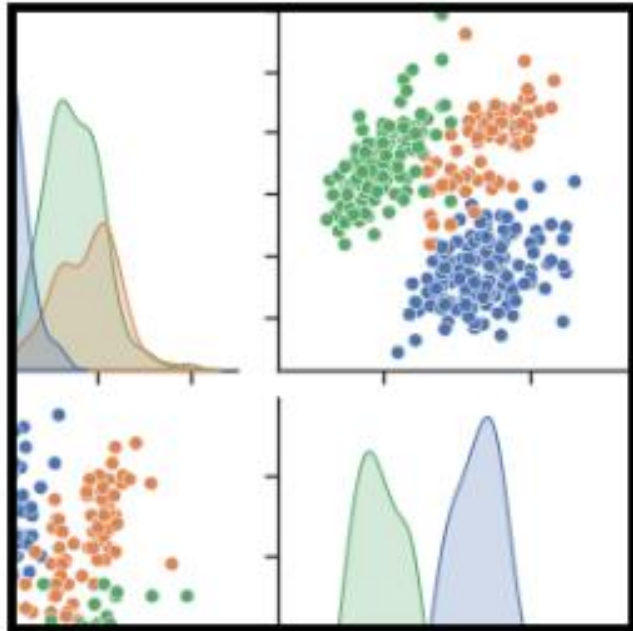
- Seaborn, Plotly와 같은 다른 최신 시각화 라이브러리에 비해 덜 세련된 그래프를 제공

03

Seaborn

✓ Seaborn

- Seaborn은 matplotlib 보다 다양한 그래프 스타일 설정을 할 수 있다는 장점이 있음



☑ Seaborn의 기능과 장점

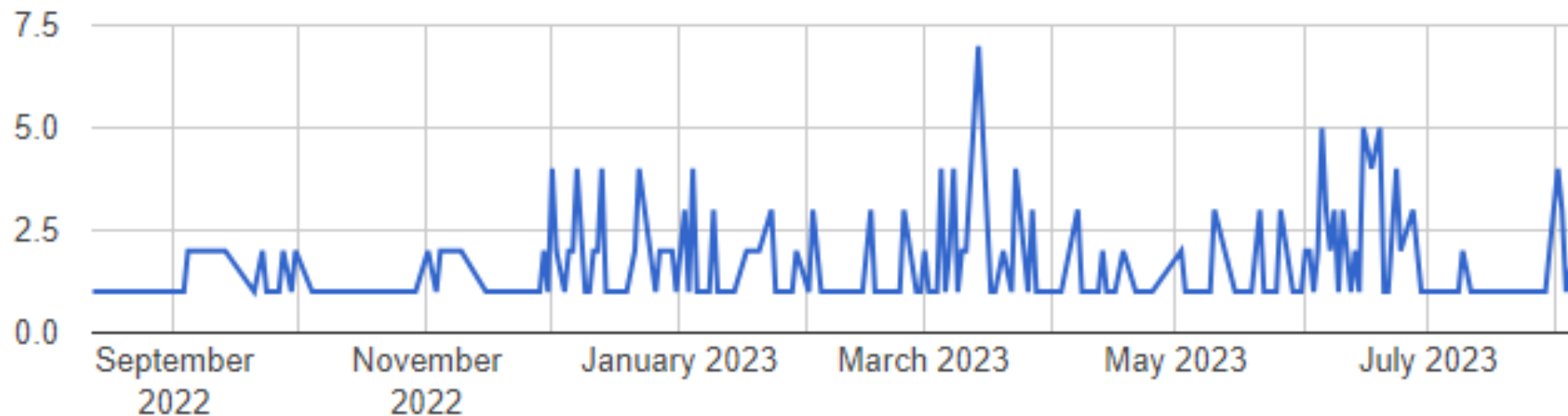
- Matplotlib 기반인 Seaborn
 - Seaborn은 Matplotlib의 기능이 확장된 버전으로, 복잡한 시각화를 더 쉽게 생성할 수 있음
- 시각화가 세련됨
 - Seaborn은 플롯을 미적으로 세련되게 시각화 하는데 사용되는 여러 테마가 포함 되어있음
- 복잡한 코드의 단순화
 - Matplotlib에 비해 복잡한 플롯을 더 간단한 구문으로 구현 가능함

☑ Seaborn의 기능과 장점

- Pandas와의 통합
 - Pandas DataFrames와 원활하게 작동하여 DataFrame에서 데이터를 직접 다루고 시각화하는 데 더 쉬움
- 고급 통계 플롯
 - 데이터 내에서 통계 관계의 다양한 측면을 보여주는 다양한 고급 플롯을 생성하는 함수가 포함

✓ Flights 데이터세트 로드

- 미국 교통통계국(Bureau of Transportation Statistics)에서 제공한 가장 포괄적인 공개된 항공 데이터세트



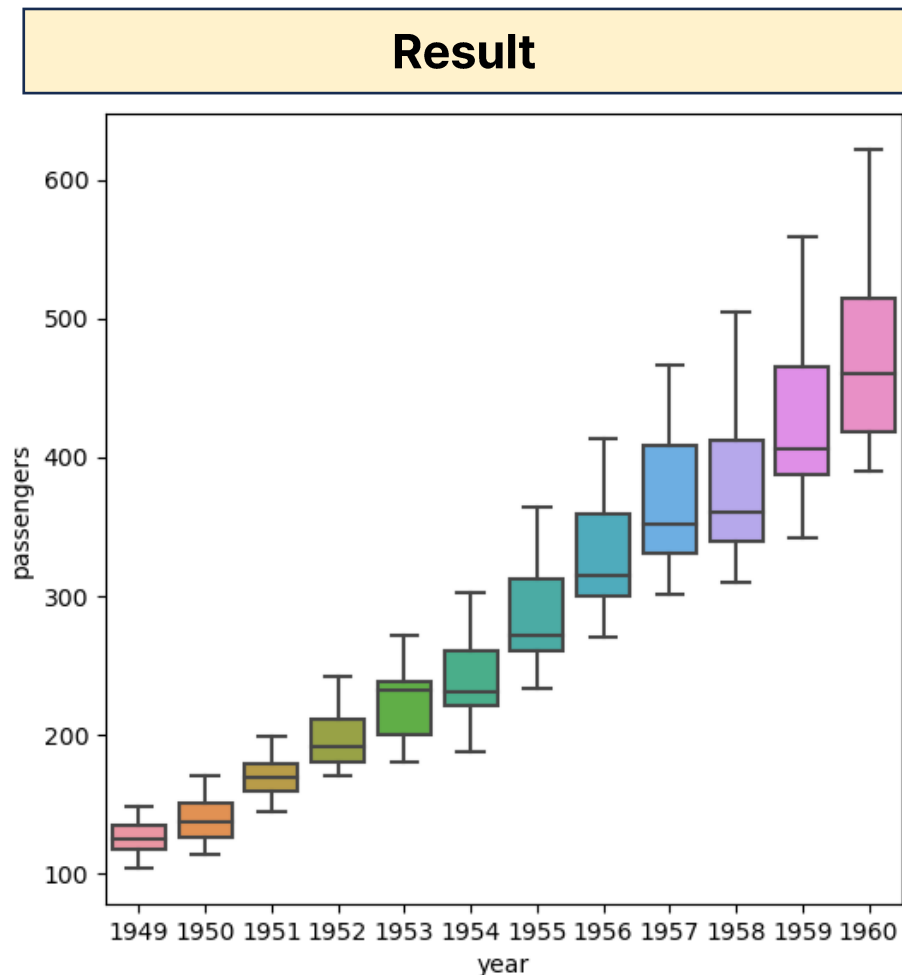
☑ Seaborn-박스 플롯(box plot)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# seaborn에서 자체 제공하는 flights 데이터셋 로드
flights = sns.load_dataset('flights')

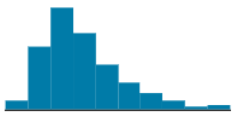
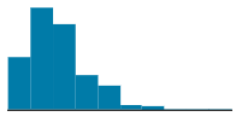
# 그래프의 크기 설정
plt.figure(figsize=(6, 6))

# 박스 플롯(box plot)으로 시각화
sns.boxplot(data=flights, x='year', y='passengers')
plt.show()
```



✓ Tips 데이터셋

- 레스토랑에 방문한 손님이 얼마나 팁을 주었는지를 성별, 흡연여부, 요일, 식사 시간과 함께 제공하는 데이터셋

# total_bill	# tip	Δ sex	✓ smoker	Δ day	Δ time
Total bill (cost of the meal), including tax, in US dollars	Tip (gratuity) in US dollars	Sex of person paying for the meal (0=male, 1=female)	Smoker in party? (0=No, 1=Yes)	3=Thur, 4=Fri, 5=Sat, 6=Sun	0=Day, 1=
		Male 64% Female 36%	true 0 0% false 0 0%	Sat 36% Sun 31% Other (81) 33%	Dinner Lunch
16.99	1.01	Female	No	Sun	Dinner
10.34	1.66	Male	No	Sun	Dinner
21.01	3.5	Male	No	Sun	Dinner
23.68	3.31	Male	No	Sun	Dinner
24.59	3.61	Female	No	Sun	Dinner
25.29	4.71	Male	No	Sun	Dinner
8.77	2	Male	No	Sun	Dinner

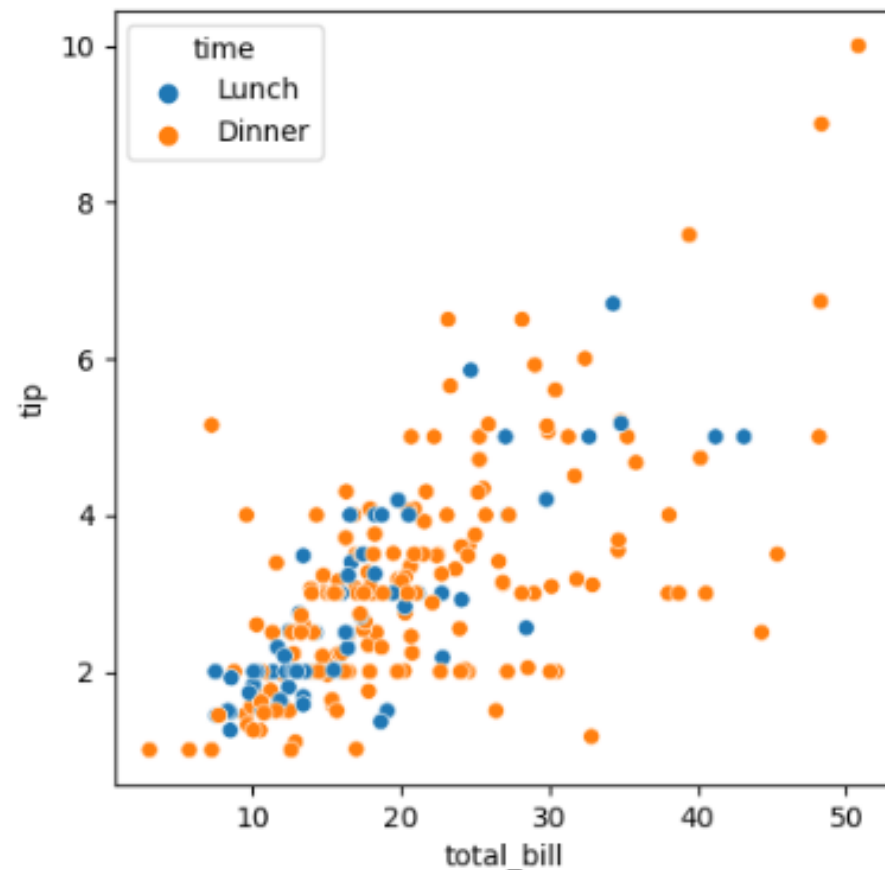
☑ Seaborn-산점도(scatter plot)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# seaborn에서 자체 제공하는 tips 데이터세트 로드
tips = sns.load_dataset("tips")

# 그래프의 크기 설정
plt.figure(figsize=(5, 5))

# 산점도(scatter plot) 그래프로 시각화
sns.scatterplot(data=tips, x="total_bill", y="tip", hue="time")
plt.show()
```

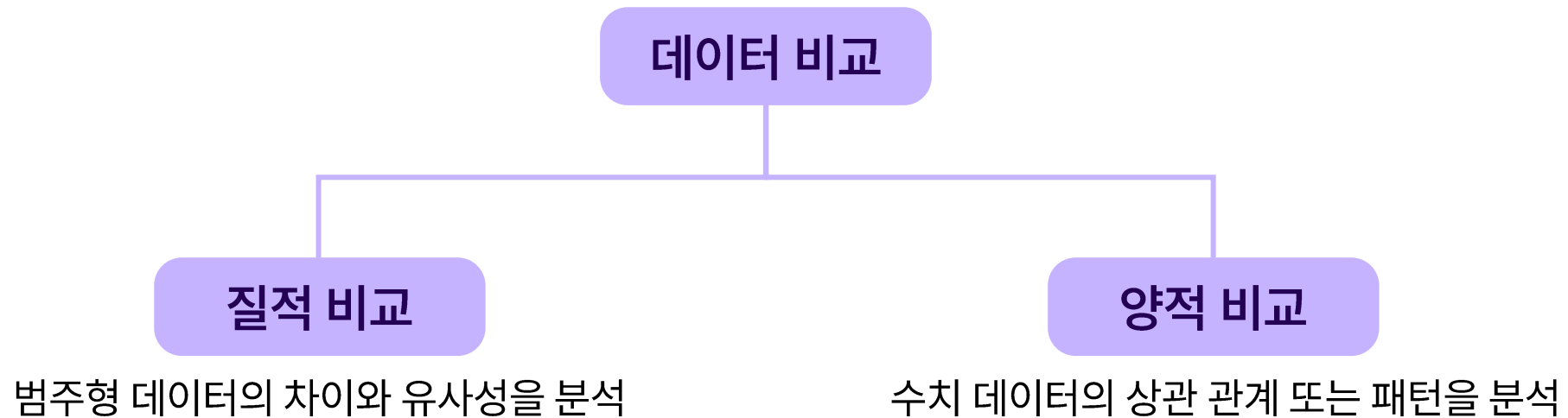
Result

04

데이터 비교

☑ 데이터 비교란?

- 두 개 이상의 데이터 집합을 분석하고 평가하여 유사성, 차이, 추세, 관계 등을 파악하는 과정



👉 Flights 데이터세트 헤드(head) 출력

- 데이터 세트를 파악하기 위해 head()를 이용해 가장 위에 5줄을 출력

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# seaborn에서 자체 제공하는 flights 데이터세트 로드
flights = sns.load_dataset('flights')
print(flights.head())
```

Result			
	year	month	passengers
0	1949	Jan	112
1	1949	Feb	118
2	1949	Mar	132
3	1949	Apr	129
4	1949	May	121

☑ 연도별 승객 수 시각화

- 막대 그래프(bar plot)을 이용해 연도별 승객을 시각화

```
import matplotlib.pyplot as plt
import seaborn as sns

# 축의 글씨 크기 변경
sns.set(font_scale=1.5)

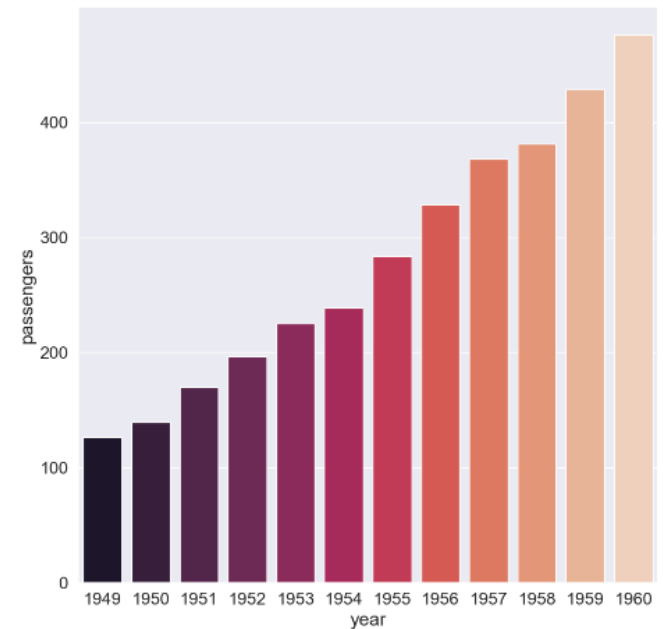
# 시각화할 그래프 크기 변경
plt.figure(figsize=(10, 10))

# 막대 그래프로 시각화
sns.barplot(data=flights, x="year", y="passengers", palette="rocket", ci=None)
plt.show()
```

```
sns.color_palette("rocket")
```



Result



✓ 연도별 승객 수 시각화

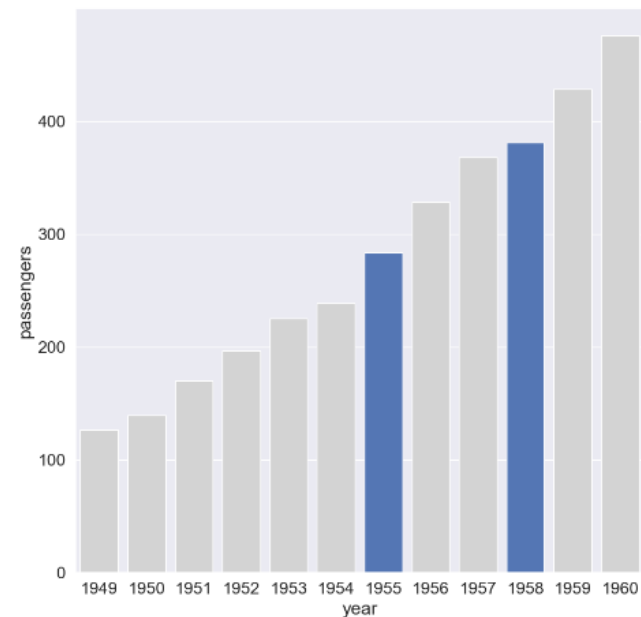
- 막대 그래프의 palette 값을 직접 생성하여 1955년과 1958년 승객 수를 비교하기 쉽게 시각화

```
# 축의 글씨 크기 변경
sns.set(font_scale=1.5)

# 시각화할 그래프 크기 변경
plt.figure(figsize=(10, 10))

# 데이터 내 연도 카테고리 리스트화
flights_years = sorted(list(set(flights['year'])))

# 막대 그래프로 시각화
color = ['lightgrey' if x not in [1955, 1958] else "#4472C4" for x in flights_years]
sns.barplot(data=flights, x="year", y="passengers", palette=color, ci=None)
plt.show()
```

Result

✓ 월별 승객 수 시각화

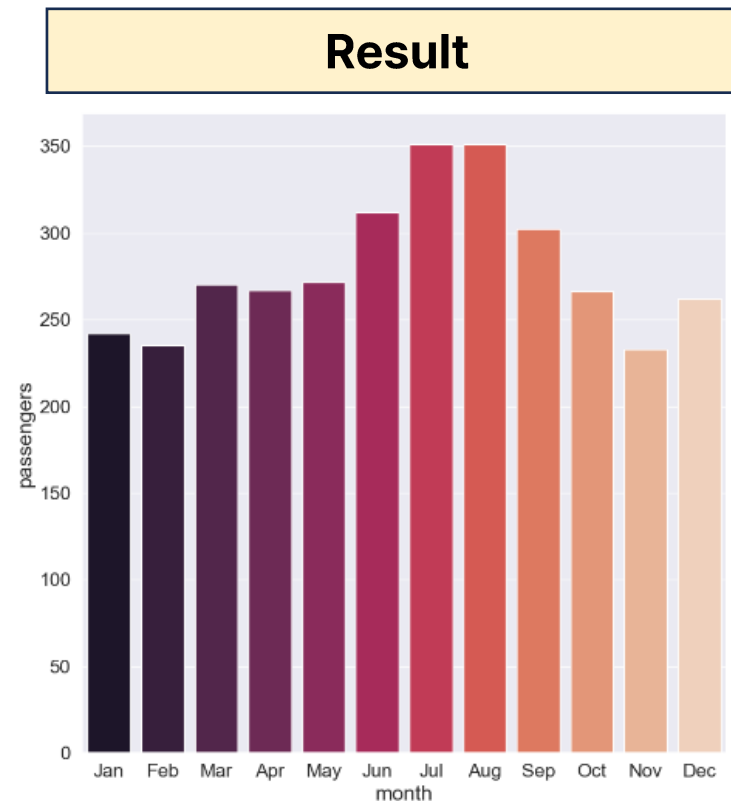
- 막대 그래프(bar plot)을 이용해 월별 승객 시각화

```
import matplotlib.pyplot as plt
import seaborn as sns

# 축의 글씨 크기 변경
sns.set(font_scale=1.5)

# 시각화할 그래프 크기 변경
plt.figure(figsize=(10, 10))

# 막대 그래프로 시각화 (x축을 연도(year)에서 월(month)로 변경)
sns.barplot(data=flights, x="month", y="passengers", palette="rocket", ci=None)
plt.show()
```



✓ 월별 승객 수 시각화

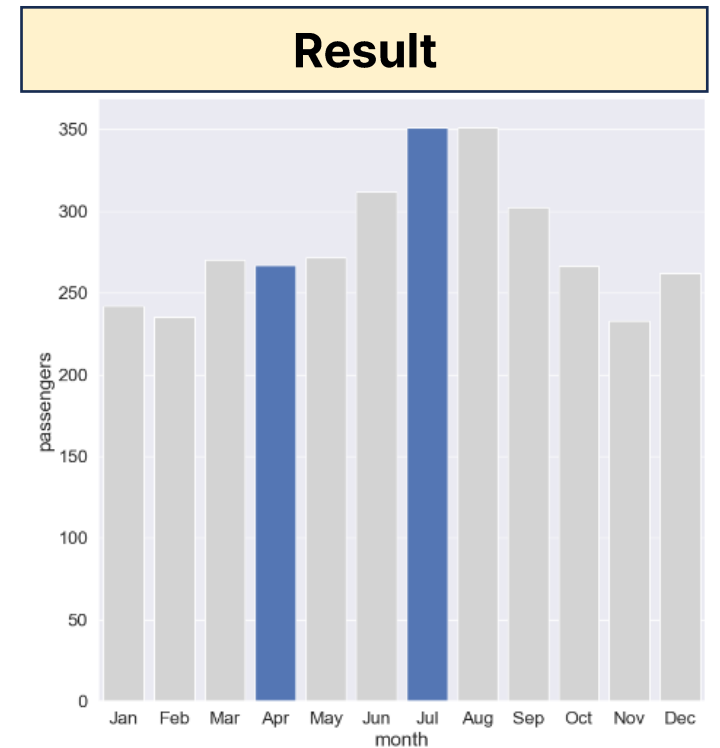
- 막대 그래프의 palette 값을 직접 생성하여 적용

```
# 데이터 내 월 카테고리 리스트화
flights_month = ['Jan', 'Feb', 'Mar', 'Apr', 'May',
                 'Jun', 'Jul', 'Aug', 'Sep',
                 'Oct', 'Nov', 'Dec']

print(flights_month)

# 막대 그래프로 시각화
color = ['lightgrey' if x not in ['Apr', 'Jul'] else "#4472C4" for x in flights_month]
sns.barplot(data=flights, x="month", y="passengers", palette=color, ci=None)

plt.show()
```

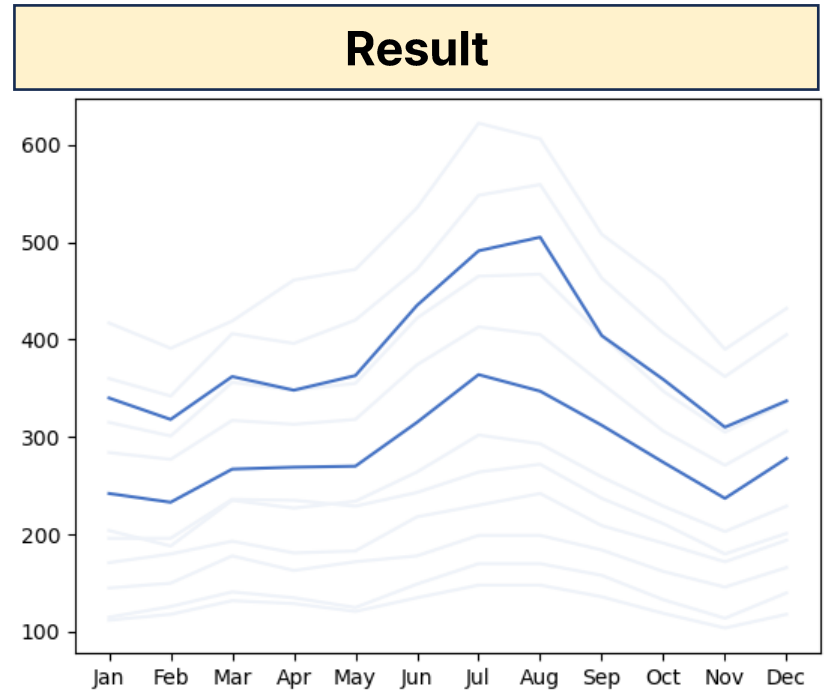


연도별 승객 수 변화 비교

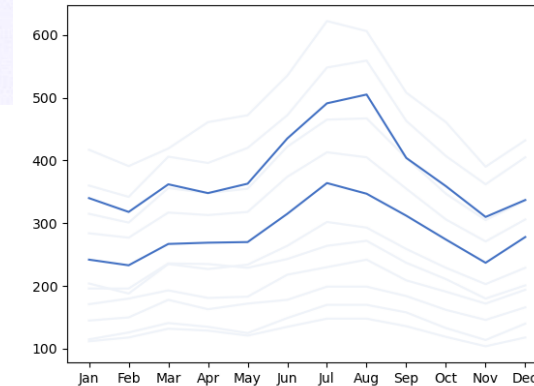
```
import matplotlib.pyplot as plt
import seaborn as sns

# 연도마다 월별 승객 수 변화 시각화
for year in flights['year'].unique():
    flights_for_year = flights[flights['year'] == year]
    # 1955년과 1958년 승객 수 변화를 파란색으로 강조
    line_color = '#4472C4' if year == 1955 or year == 1958 else '#FF3F9'
    plt.plot(flights_for_year['month'], flights_for_year['passengers'], color=line_color)

plt.show()
```



연도별 승객 수 변화 비교



flights["year"].unique() = [1949 1950 1951 1952 1953 1954 1955 1956 1957 1958 1959 1960]

year = 1949일 때,

flights_for_year =

	year	month	passengers
0	1949	Jan	112
1	1949	Feb	118
2	1949	Mar	132
3	1949	Apr	129
4	1949	May	121
5	1949	Jun	135
6	1949	Jul	148
7	1949	Aug	148
8	1949	Sep	136
9	1949	Oct	119
10	1949	Nov	104
11	1949	Dec	118

```
import matplotlib.pyplot as plt
import seaborn as sns

# 연도마다 월별 승객 수 변화 시각화
for year in flights['year'].unique():
    flights_for_year = flights[flights['year'] == year]
    # 1955년과 1958년 승객 수 변화를 파란색으로 강조
    line_color = '#4472C4' if year == 1955 or year == 1958 else '#FF3F9'
    plt.plot(flights_for_year['month'], flights_for_year['passengers'], color=line_color)

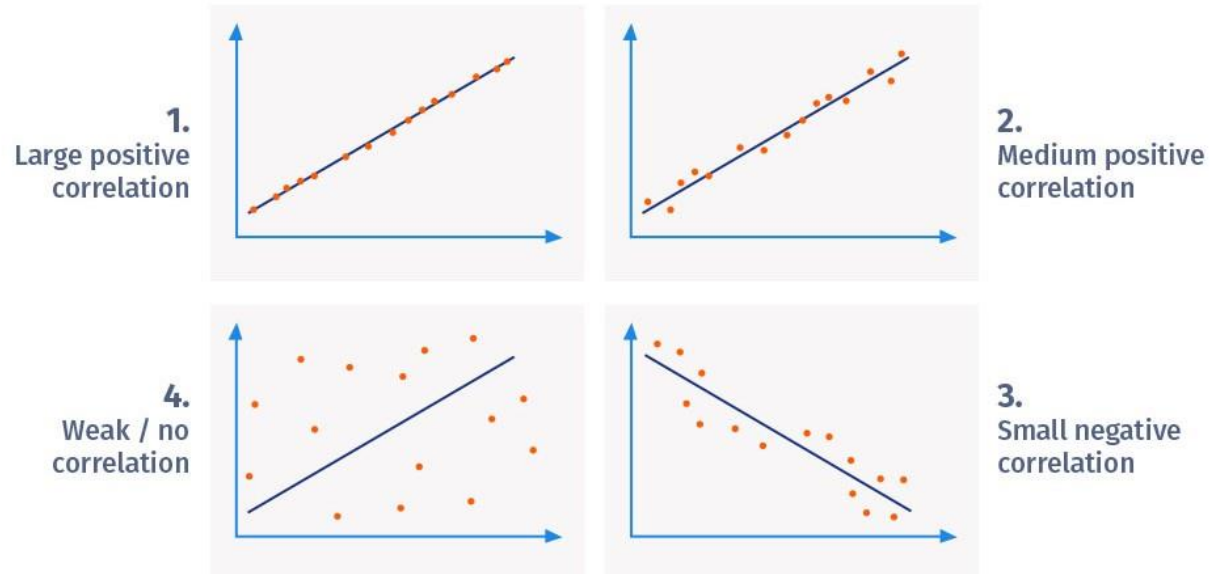
plt.show()
```

05

상관관계 분석

☑ 상관관계 분석이란?

- 두 양적 변수 간의 선형 관계의 강도와 방향을 평가하는 것을 의미
- 예를 들어, 키와 몸무게의 관계를 보자면 각각 두 변수는 선형적 상관(correlation)이 있다고 할 수 있음



✓ 공분산

- 두 변수의 공동 변동성을 설명하는 척도
- 두 변수가 어떻게 함께 변하는지를 이해하는 데 도움이 될 수 있음
- 두 변수가 함께 증가하거나 감소하면 공분산은 양수가 되며, 하나의 변수가 증가할 때 다른 변수가 감소하면 공분산은 음수가 됨

$$Cov(X, Y) = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{n}$$

☑ 공분산

- 넘파이의 내장함수 cov를 이용하면 쉽게 공분산 계산 가능

```
import pandas as pd
import numpy as np

df = pd.DataFrame({'키': [158, 163, 170, 183, 180], '몸무게': [49, 52, 60, 80, 83]})
print(np.cov(df['키'], df['몸무게']))
```

	키	몸무게
0	158	49
1	163	52
2	170	60
3	183	80
4	180	83

Result	
[[114.7 164.7] [164.7 249.7]]	



	키	몸무게
키	114.7	164.7
몸무게	164.7	249.7

✓ 공분산의 한계

- 넘파이의 내장함수 cov를 이용하면 쉽게 공분산 계산 가능

	키 (cm)	몸무게
0	158	49
1	163	52
2	170	60
3	183	80
4	180	83

	키	몸무게
키	114.7	164.7
몸무게	164.7	249.7

	키 (m)	몸무게
0	1.58	49
1	1.63	52
2	1.70	60
3	1.83	80
4	1.80	83

	키	몸무게
키	0.01147	1.647
몸무게	1.647	249.7

✓ 상관관계

```
import pandas as pd
import numpy as np

df = pd.DataFrame({'키':[158, 163, 170, 183, 180], '몸무게':[49, 52, 60, 80, 83]})
print(np.corrcoef(df['키'],df['몸무게'])[0][1])
print('-'*25)
df2 = pd.DataFrame({'x':[1.58, 1.63, 1.70, 1.83, 1.80], 'y':[49, 52, 60, 80, 83]})
print(np.corrcoef(df2['x'],df2['y'])[0][1])
```

Result

0.9732011625327036

0.9732011625327038

06

상관관계 시각화

✓ 상관관계 시각화의 장점

- 두 변수 사이의 관계의 강도와 방향을 식별하는 데 도움이 됨
- 데이터에 대한 통찰력을 얻을 수 있어 필요한 특징 선택에 도움이 됨

☑ 당뇨병(diabetes) 데이터셋

- Scikit-learn에서 제공하는 회귀 작업에 자주 사용되는 표준 데이터셋 중 하나

```
import pandas as pd
import numpy as np

from sklearn import datasets

# 당뇨병 데이터셋을 로드합니다.
dataset = datasets.load_diabetes()

# 데이터셋 내 카테고리들을 확인합니다.
print(data.keys())

# 데이터셋이 포함하는 특성들을 확인합니다.
print(data['feature_names'])
```

Result

```
dict_keys(['data', 'target', 'frame',
['age', 'sex', 'bmi', 'bp', 's1', 's2']
```

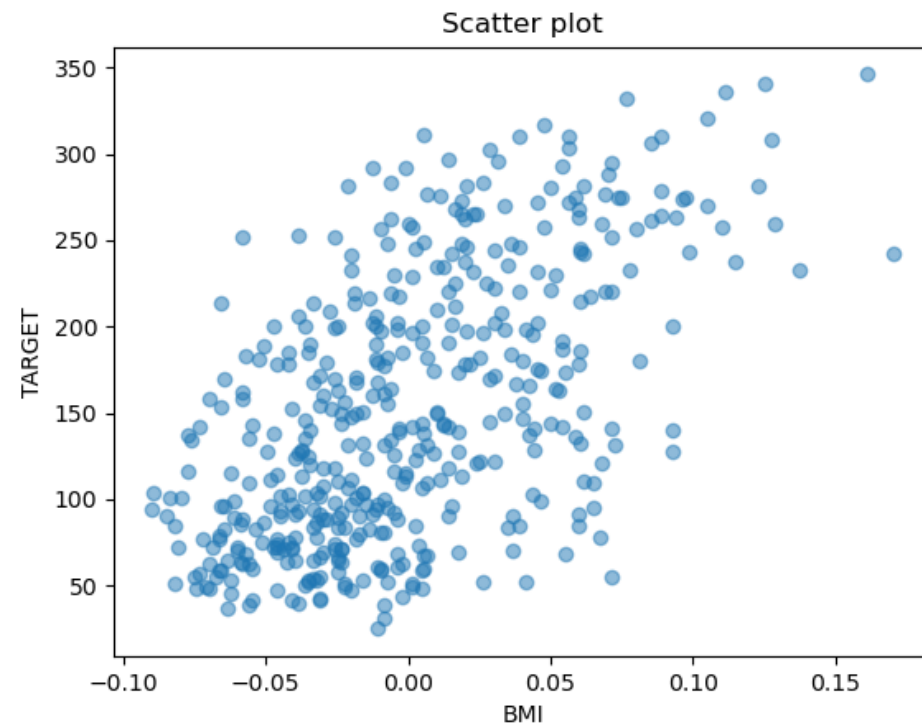
④ 산점도(scatter plot)

```
import matplotlib.pyplot as plt  
plt.scatter(X, Y, alpha=0.5)  
plt.title("Scatter plot")
```

```
plt.xlabel('BMI')  
plt.ylabel('TARGET')
```

```
# 산점도 그래프 시각화  
plt.show()
```

Result

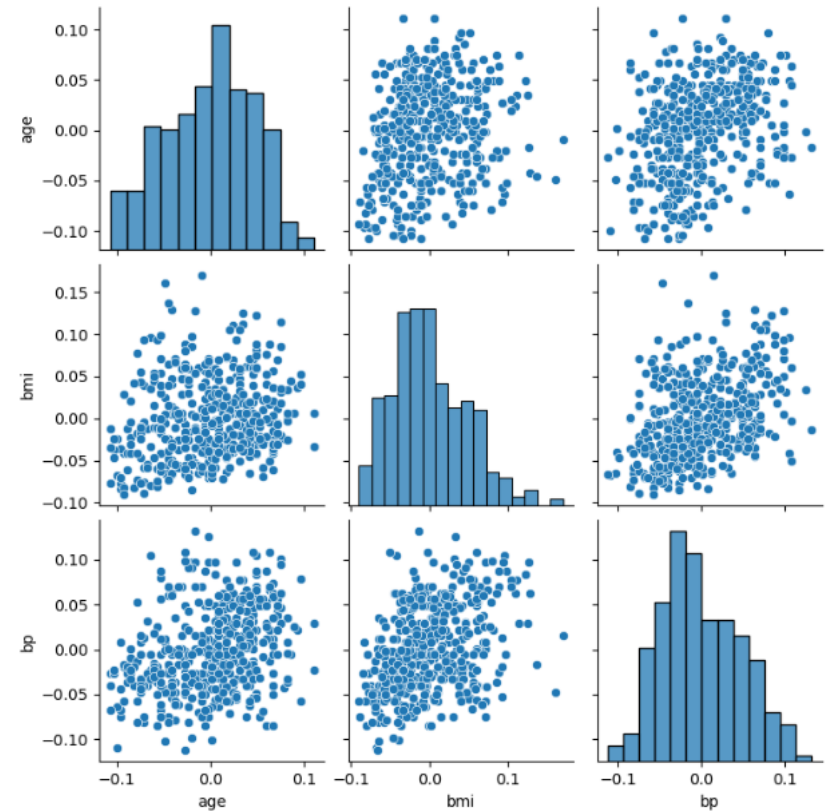


☑ 여러 변수간 산점도(pairplot)

```
import pandas as pd
from sklearn import datasets
import matplotlib.pyplot as plt
import seaborn as sns

data = datasets.load_diabetes()
df = pd.DataFrame(data['data'], index=data['target'], columns=data['feature_names'])
# 행에서 중복된 데이터 제거
df = df.reset_index(drop=True)
plt.figure(figsize=(14, 8))
fig = sns.pairplot(df[['age', 'bmi', 'bp']])
plt.show()
```

Result



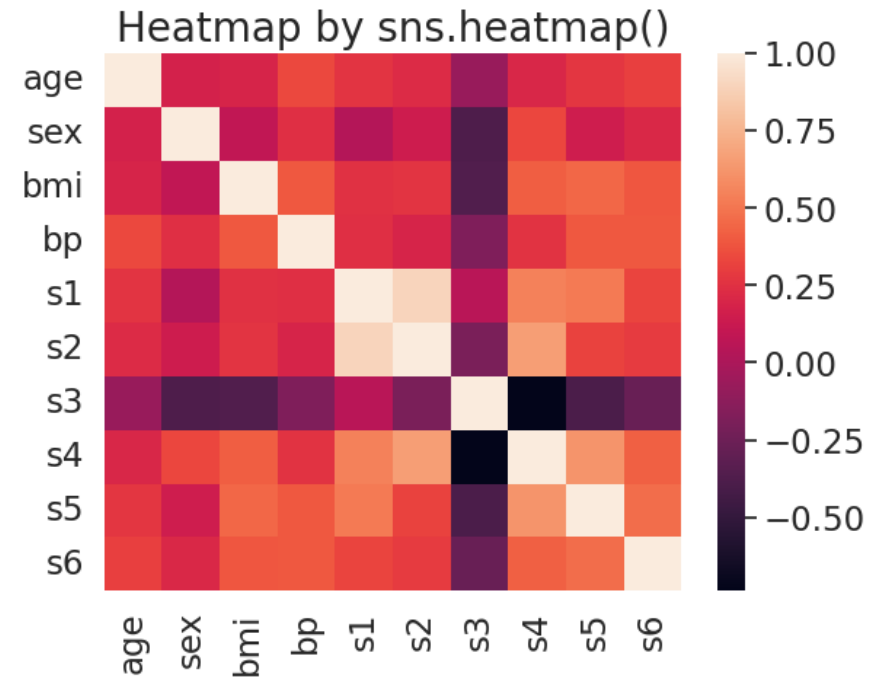
✔ 히트맵(heatmap)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets

data = datasets.load_diabetes()
df = pd.DataFrame(data['data'], index=data['target'], columns=data['feature_names'])
df = df.reset_index(drop=True)

sns.heatmap(df.corr())
plt.title('Heatmap by sns.heatmap()', fontsize=20)
plt.show()
```

Result



✔ 히트맵(heatmap) with matplotlib

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets

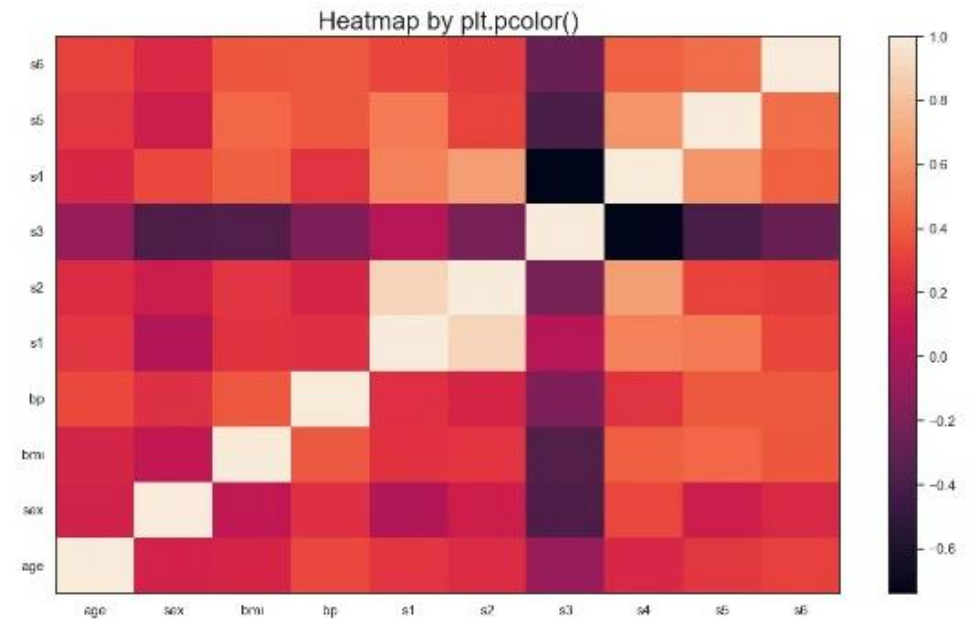
data = datasets.load_diabetes()
df = pd.DataFrame(data['data'], index=data['target'], columns=data['feature_names'])
df = df.reset_index(drop=True)

plt.pcolor(df.corr())

plt.xticks(np.arange(0.5, len(df.columns), 1), df.columns)
plt.yticks(np.arange(0.5, len(df.columns), 1), df.columns)
plt.title('Heatmap by plt.pcolor()', fontsize=20)

plt.colorbar()
plt.show()
```

Result

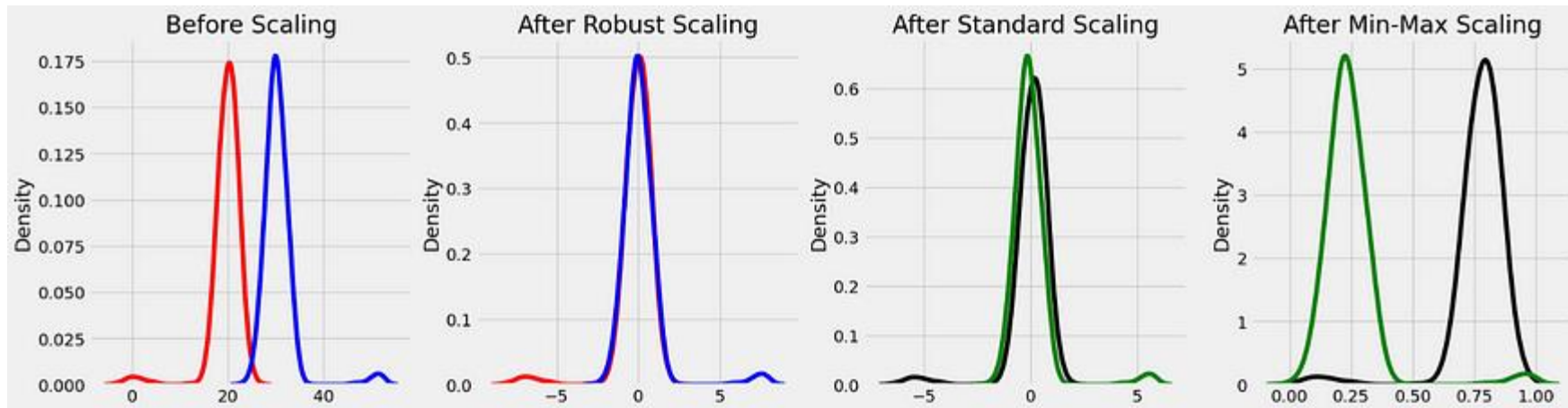


07

데이터 스케일링

☑ 데이터 스케일링

- 서로 다른 변수의 값의 범위가 일치하도록 조정하는 작업



☑ 데이터 스케일링의 필요성

	주택 크기	수영장의 수	주택 가격
0	9000	3	300,000,000
1	33	1	100,000
2	1253	2	20,000,000
3	240	2	990,000
4	123	1	800,000

- 모든 변수를 동일한 범위가 되기 때문에 쉽게 비교할 수 있게 만듦
- 규모가 큰 변수의 영향이 그러지 못한 변수보다 크게 작용되는 것을 방지
- 이상치를 처리하는데 효과적

④ 데이터 스케일링의 종류

- Min-Max 스케일링
 - 변수의 값을 변수의 최소값과 최대값을 사용하여 0과 1사이의 값으로 변환
- 표준화:
 - 변수의 값을 평균이 0이고 표준 편차가 1이 되도록 변환
- 로버스트 스케일링
 - 변수의 값을 중앙값이 0이고 사분위범위(IQR)가 1이 되도록 변환

☑ 암(Cancer) 데이터세트

- 데이터 내 특징들의 값이 크게 상이한 데이터세트로, 스케일링 연습에 적합
- 최솟값: 0, 최댓값: 4254

	mean radius	mean texture	...	worst fractal dimension	target
0	17.99	10.38	...	0.11890	0
1	20.57	17.77	...	0.08902	0
2	19.69	21.25	...	0.08758	0
3	11.42	20.38	...	0.17300	0
4	20.29	14.34	...	0.07678	0

✔ 표준화

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler

cancer = load_breast_cancer()
X = cancer.data

std = StandardScaler()
std.fit(X)
X_scaled = std.transform(X)

print('Data before scaling')
print(X, '\n')

print('Data after scaling')
print(X_scaled)
```

Result

Data before scaling

```
[[1.799e+01 1.038e+01 1.228e+02 ... 2.654e-01 4.601e-01 1.189e-01]
 [2.057e+01 1.777e+01 1.329e+02 ... 1.860e-01 2.750e-01 8.902e-02]
 [1.969e+01 2.125e+01 1.300e+02 ... 2.430e-01 3.613e-01 8.758e-02]
 ...
 [1.660e+01 2.808e+01 1.083e+02 ... 1.418e-01 2.218e-01 7.820e-02]
 [2.060e+01 2.933e+01 1.401e+02 ... 2.650e-01 4.087e-01 1.240e-01]
 [7.760e+00 2.454e+01 4.792e+01 ... 0.000e+00 2.871e-01 7.039e-02]]
```

Data after scaling

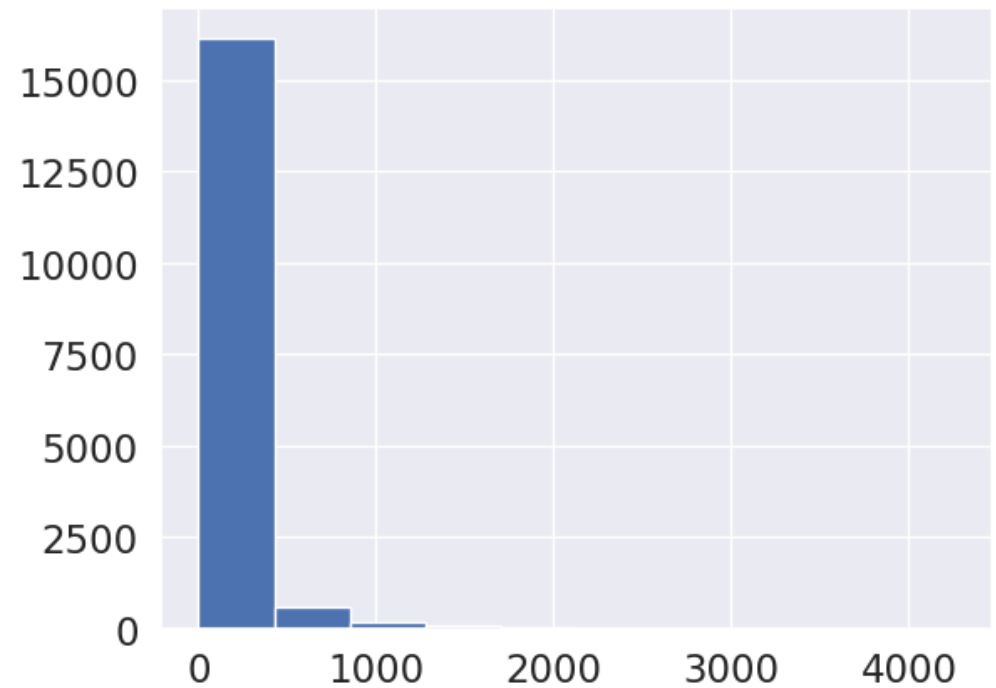
```
[[ 1.09706398 -2.07333501  1.26993369 ...  2.29607613  2.75062224
   1.93701461]
 [ 1.82982061 -0.35363241  1.68595471 ...  1.0870843  -0.24388967
   0.28118999]
 [ 1.57988811  0.45618695  1.56650313 ...  1.95500035  1.152255
   0.20139121]
 ...
 [ 0.70228425  2.0455738  0.67267578 ...  0.41406869 -1.10454895
  -0.31840916]
 [ 1.83834103  2.33645719  1.98252415 ...  2.28998549  1.91908301
   -0.00000000]]
```

✓ 표준화

```
import matplotlib.pyplot as plt

X_reshape = X.reshape(17070, 1)
plt.hist(X_reshape)
plt.show()
```

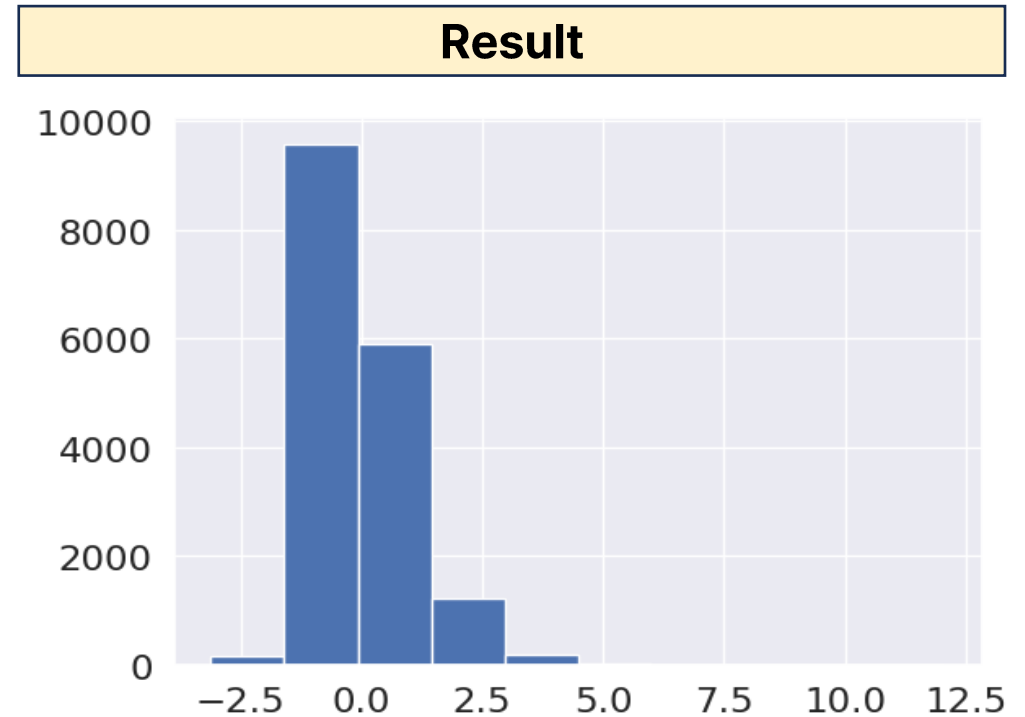
Result



✓ 표준화

```
import matplotlib.pyplot as plt

X_scaled_reshape = X_scaled.reshape(17070, 1)
plt.hist(X_scaled_reshape)
plt.show()
```



☑ Min-Max 스케일링

```
from sklearn.preprocessing import MinMaxScaler
mms = MinMaxScaler()
mms.fit(X)

# Min-Max 스케일링
X_scaled = mms.transform(X)

print("Data before scaling!")
print(X, '\n')

print("Data After scaling!")
print(X_scaled)
```

Result

```
Data before scaling!
[[1.799e+01 1.038e+01 1.228e+02 ... 2.654e-01 4.601e-01
 [2.057e+01 1.777e+01 1.329e+02 ... 1.860e-01 2.750e-01
 [1.969e+01 2.125e+01 1.300e+02 ... 2.430e-01 3.613e-01
```

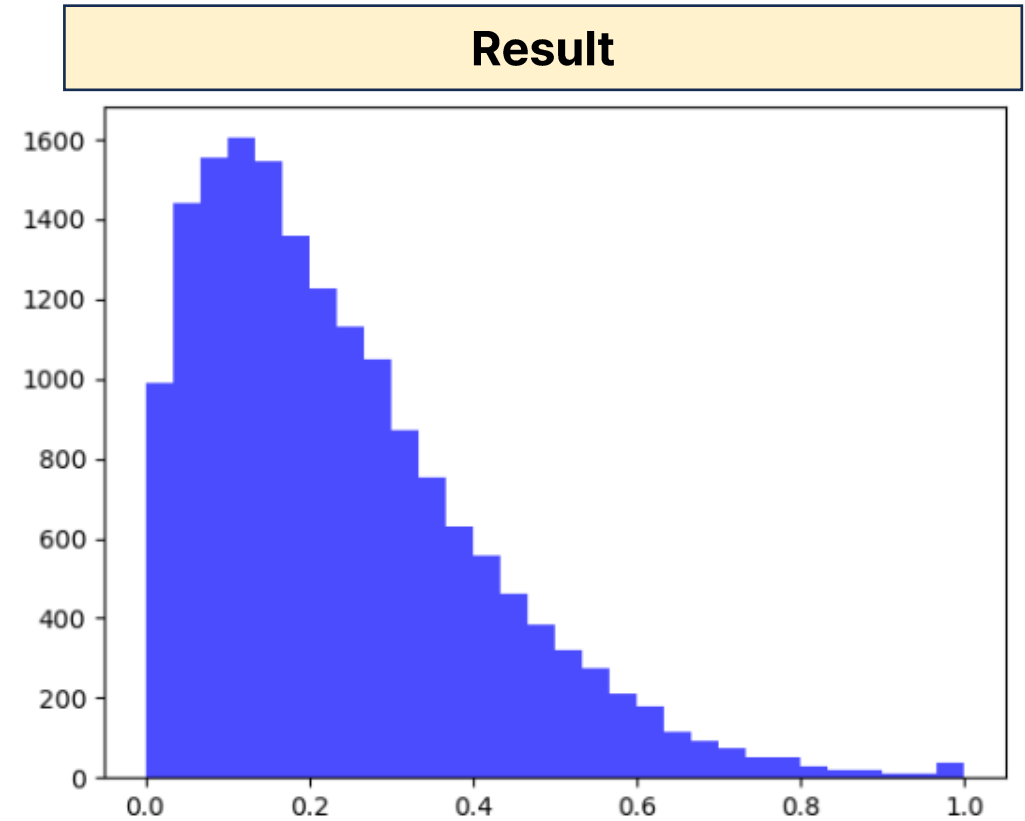
Result

```
Data After scaling!
[[0.52103744 0.0226581 0.54598853 ... 0.91202749 0.59846245
 [0.64314449 0.27257355 0.61578329 ... 0.63917526 0.23358959
 [0.60149557 0.3902604 0.59574321 ... 0.83505155 0.40370589
 ...
```

✓ Min-Max 스케일링

```
import matplotlib.pyplot as plt
X_scaled_reshape = X_scaled.reshape(17070, 1)
plt.hist(X_scaled_reshape, bins=30, color='blue', alpha=0.7)
# 스케일링 후 데이터분포 시각화

plt.show()
```



✓ 로버스트 스케일링

```
from sklearn.preprocessing import RobustScaler

rbs = RobustScaler()
rbs.fit(X)
X_scaled = rbs.transform(X)

print('Data before scaling')
print(X, '\n')

print('Data after scaling')
print(X_scaled)
```

Result

Data before scaling!

```
[[1.799e+01 1.038e+01 1.228e+02 ... 2.654e-01 4.601e-01
 [2.057e+01 1.777e+01 1.329e+02 ... 1.860e-01 2.750e-01
 [1.969e+01 2.125e+01 1.300e+02 ... 2.430e-01 3.613e-01
```

Result

Data After scaling!

```
[[ 1.13235294 -1.5026643  1.26374006 ...  1.71524826
 1.88457808]
 [ 1.76470588 -0.19005329  1.61285862 ...  0.89219446
 0.43549952]
 [ 1.54901961  0.42806394  1.51261666 ...  1.48305173
 0.3656644 ]
```

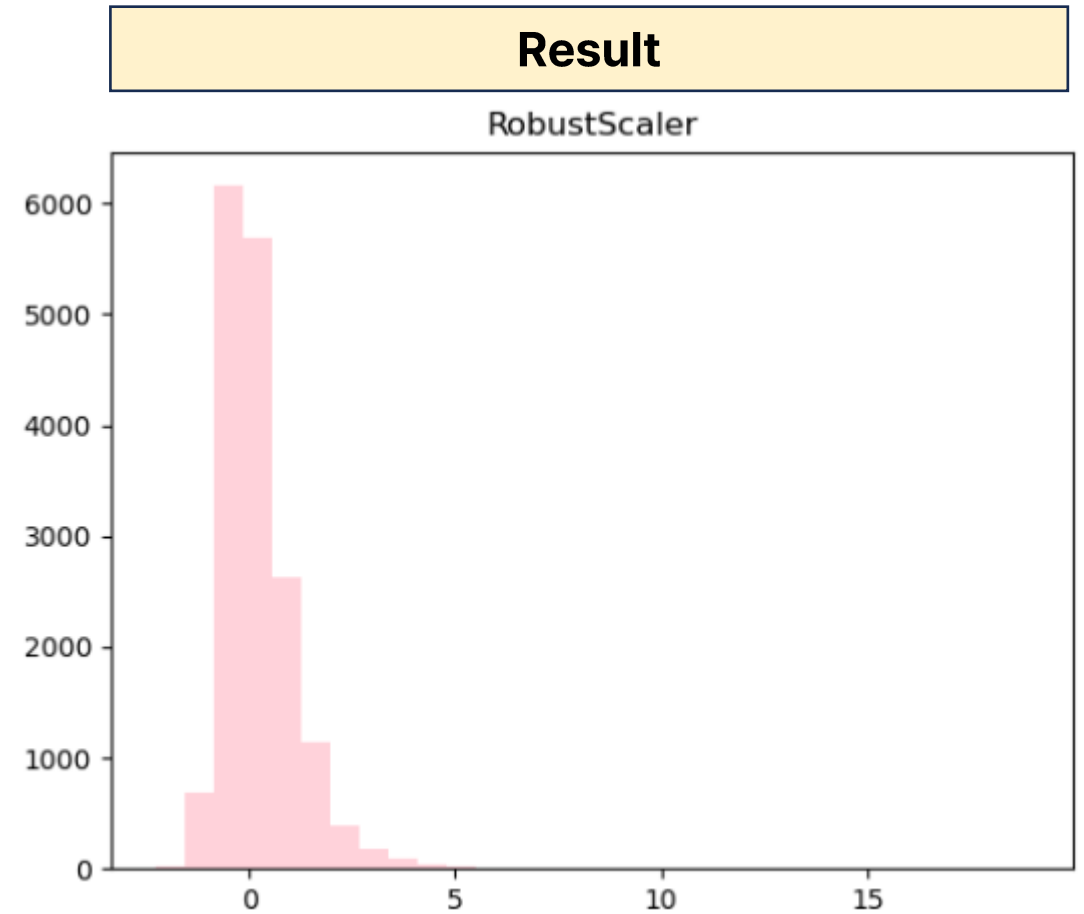
✓ 로버스트 스케일링

```
import matplotlib.pyplot as plt

# 히스토그램 시각화
plt.hist(X_scaled_reshape, bins=30, color='pink', alpha = 0.7)

plt.title('RobustScaler')

plt.show()
```



08

데이터 전처리

✓ 데이터 전처리

- 데이터를 분석하기에 효과적인 형식으로 정리, 변환, 구성하는 작업을 의미

④ 데이터 전처리

- 데이터 정제
 - 누락된 값, 잡음, 일관성 없는 데이터를 처리
- 특징 엔지니어링
 - 기존의 특징에서 새로운 특징을 생성하거나 변환하는 과정
- 불균형 데이터 처리
 - 분류 작업에서 한 클래스가 다른 클래스보다 크게 우세하면 모델에 편향을 일으킬 수 있음
 - 클래스를 균형 있게 하기 위해 오버샘플링, 언더샘플링과 같은 기술을 사용

④ 데이터 전처리

```
import seaborn as sns
titanic = sns.load_dataset('titanic')
print(titanic.info())
```

Result			
#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	survived	891 non-null	int64
1	pclass	891 non-null	int64
2	sex	891 non-null	object
3	age	714 non-null	float64
4	sibsp	891 non-null	int64
5	parch	891 non-null	int64
6	fare	891 non-null	float64
7	embarked	889 non-null	object
8	class	891 non-null	category
9	who	891 non-null	object
10	adult_male	891 non-null	bool
11	deck	203 non-null	category
12	embark_town	889 non-null	object
13	alive	891 non-null	object
14	alone	891 non-null	bool

④ 불필요한 열 제외

```
# 누락된 데이터가 많은 열 제거  
cols = ['deck']  
titanic = titanic.drop(cols, axis=1)  
print(titanic.info())
```

Result			
#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	survived	891 non-null	int64
1	pclass	891 non-null	int64
2	sex	891 non-null	object
3	age	714 non-null	float64
4	sibsp	891 non-null	int64
5	parch	891 non-null	int64
6	fare	891 non-null	float64
7	embarked	889 non-null	object
8	class	891 non-null	category
9	who	891 non-null	object
10	adult_male	891 non-null	bool
11	embark_town	889 non-null	object
12	alive	891 non-null	object
13	alone	891 non-null	bool

④ 선형보간

```
# 누락된 데이터를 선형보간을 이용해 메꿈
titanic['age'] = titanic['age'].interpolate()
print(titanic.info())
```

Result			
#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	survived	891 non-null	int64
1	pclass	891 non-null	int64
2	sex	891 non-null	object
3	age	891 non-null	float64
4	sibsp	891 non-null	int64
5	parch	891 non-null	int64
6	fare	891 non-null	float64
7	embarked	889 non-null	object
8	class	891 non-null	category
9	who	891 non-null	object
10	adult_male	891 non-null	bool
11	embark_town	889 non-null	object
12	alive	891 non-null	object
13	alone	891 non-null	bool

09

특성 엔지니어링

✓ 특성 엔지니어링

- 기존 특성에서 새로운 특성을 선택, 변환, 생성하여 기계 학습 모델의 성능을 향상시키는 과정
- 데이터 전처리의 중요한 측면이며, 알고리즘의 효과를 향상시키는 데 중요함

☑ 특성 엔지니어링 방법

- 특성 결합
 - 두 개 이상의 기존 특성을 결합하여 새로운 특성을 만들
- 범주형 변수 인코딩
 - 수치형 특성으로 변환
- 일변량 비선형 변환
 - 제곱 항 또는 세제곱항 추가
- 새로운 특성 생성
 - 사전 지식 등을 통해 기존 특성을 이용해 유용한 새로운 특성을 생성

☑ 범주형 변수 인코딩 및 특성 결합

```
# 생존 특성을 0과 1로 변경하여 상관관계를 구해보십시오.  
titanic['alive_1'] = np.where(titanic['alive']=='yes', 1, 0)  
titanic_sub = titanic[['fare', 'alive_1']]  
print(titanic_sub.corr())
```

```
print(titanic['alive'].value_counts())
```

Result

```
no      549  
yes     342  
Name: alive, dtype: int64
```

Result

```
      fare  alive_1  
fare  1.000000  0.257307  
alive_1 0.257307  1.000000
```

✓ 새로운 특징 생성

```
# 표값이 50을 넘으면 비싼표를 의미하는 1로 아니면 0으로 하여 새로운 칼럼을 추가합니다.
titanic['expensive'] = np.where(titanic['fare']>50, 1, 0)

# 마찬가지로 퀸즈타운이면 오세아니아로 아니면 유럽으로하여 승객의 출신 대륙을 표시하는
칼럼을 추가합니다.
titanic['continent'] = np.where(titanic['embark_town']=='Queenstown', 'Oceania', 'Europe')

print(titanic[['expensive', 'continent']])
```

```
print(titanic['embark_town'].value_counts())
```

Result	
Southampton	644
Cherbourg	168
Queenstown	77
Name: embark_town, dtype: int64	

Result		
	expensive	continent
0	0	Europe
1	1	Europe
2	0	Europe
3	1	Europe
4	0	Europe
..	...	...
886	0	Europe
887	0	Europe
888	0	Europe
889	0	Europe
890	0	Oceania